PERSONAL FINANCE TRACKER

track subscriptions and compute spending analytics

PROBLEM & MOTIVATVION

Poor visibility of recurring spending

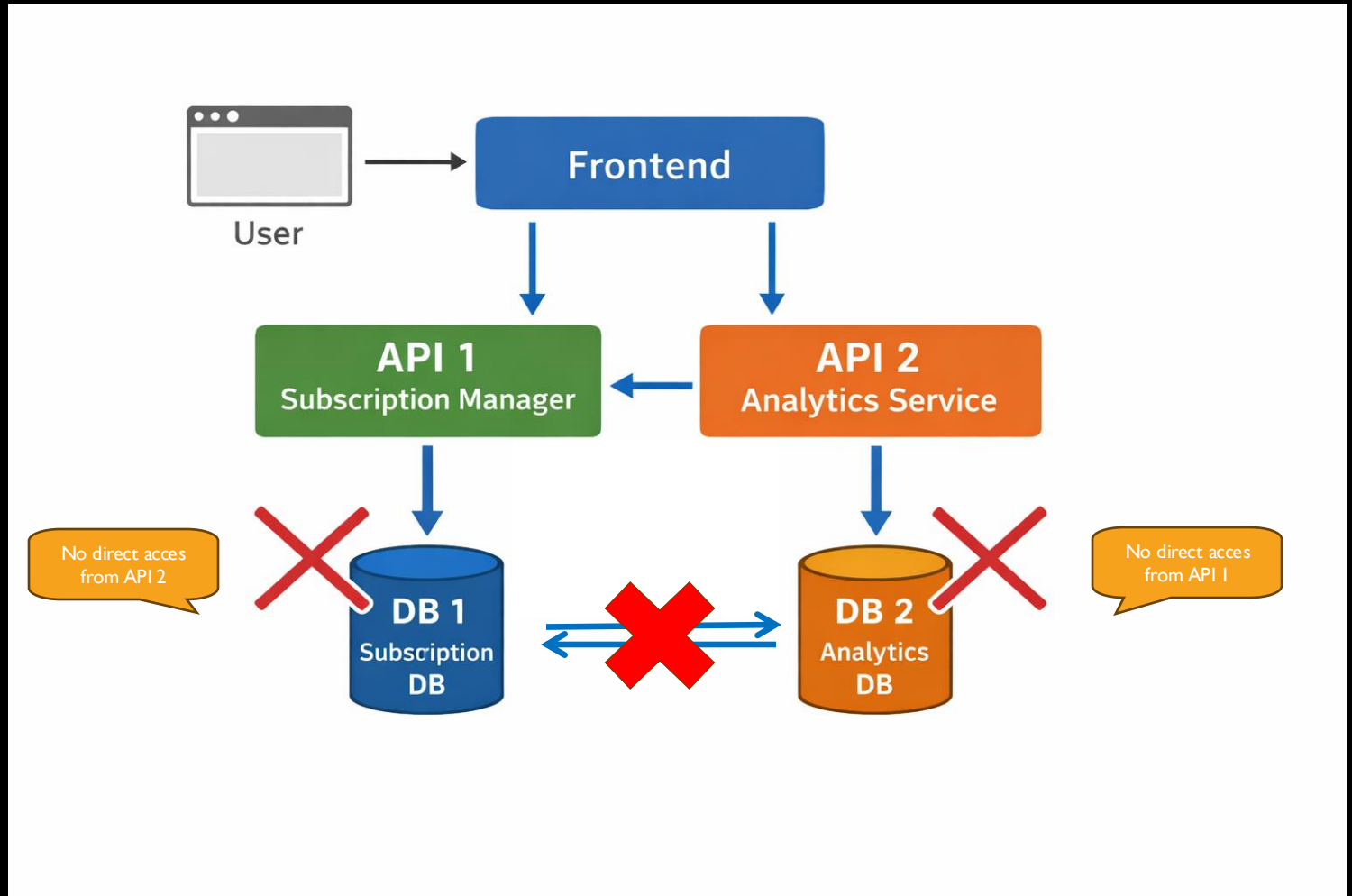
Effective financial management

improved analytics and scalability.

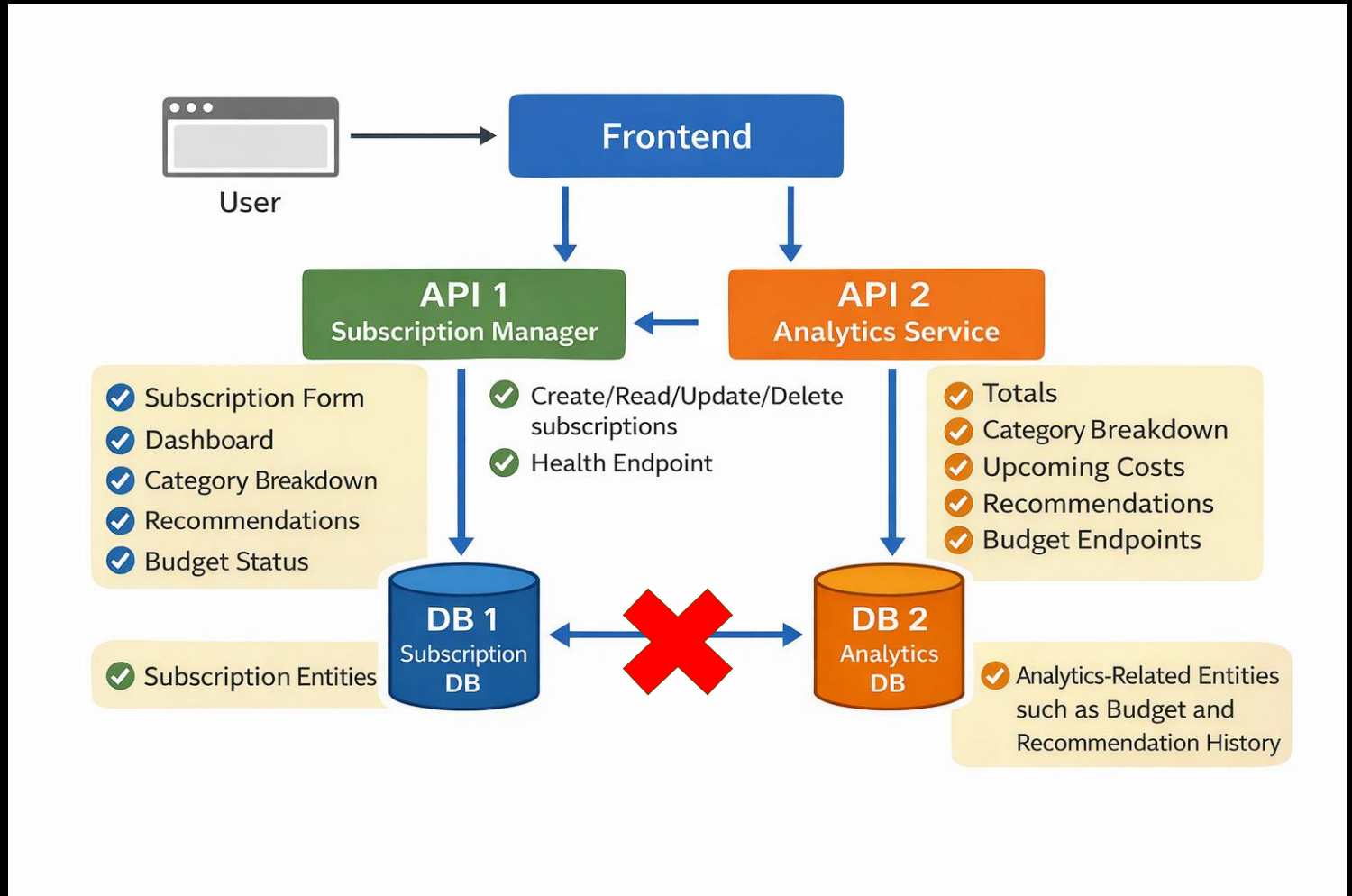
PROJECT OVERVIEW: MICROSERVICES ARCHITECTURE

- Frontend: React + Vite user interface.
- API1: Subscription Manager (FastAPI + PostgreSQL).
- API2: Analytics API (FastAPI + PostgreSQL + HTTP calls to API1).
- Docker Compose orchestrates all services.
- CI pipeline builds, scans, and publishes images.

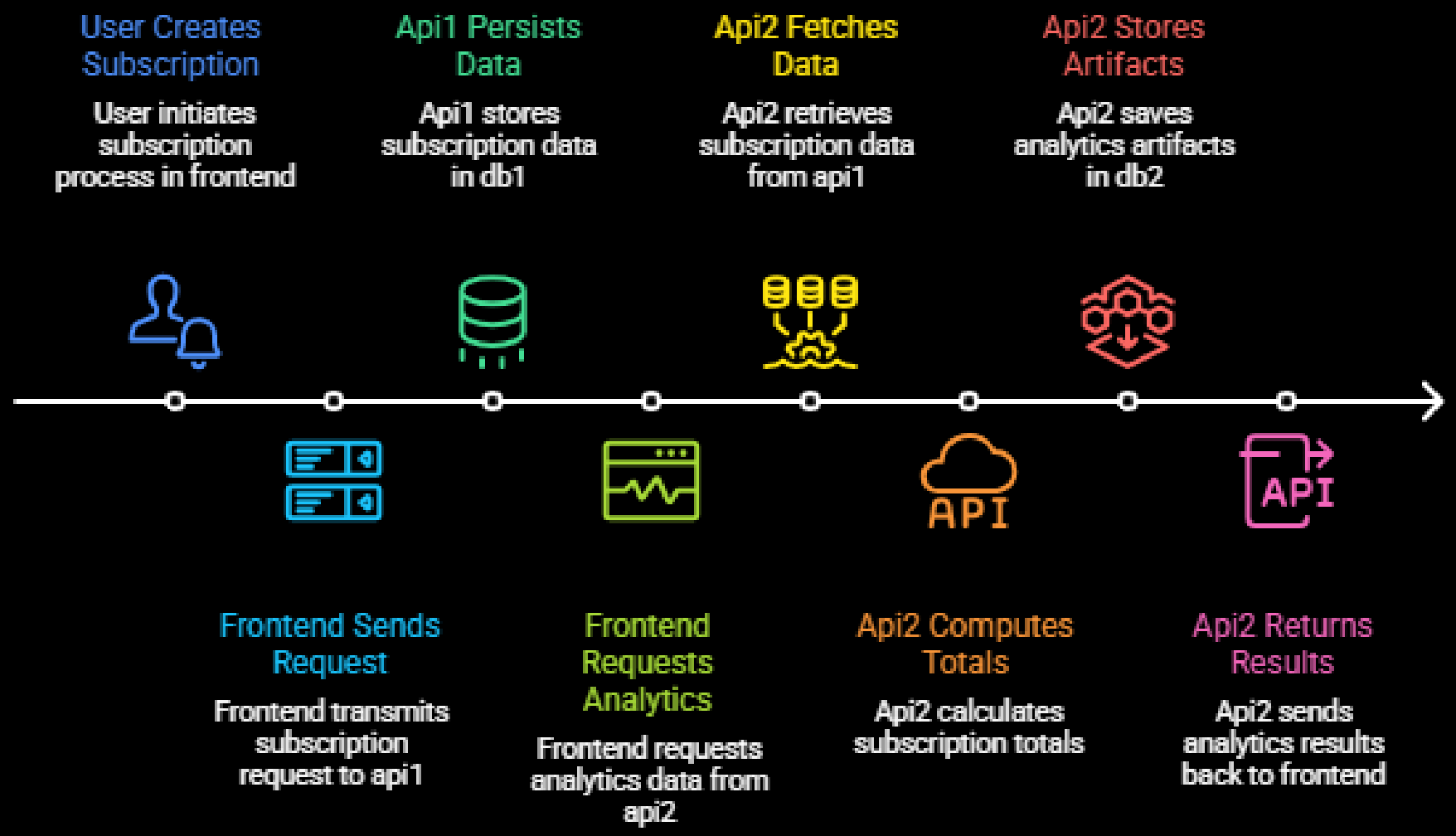
GLOBAL ARCHITECTURE



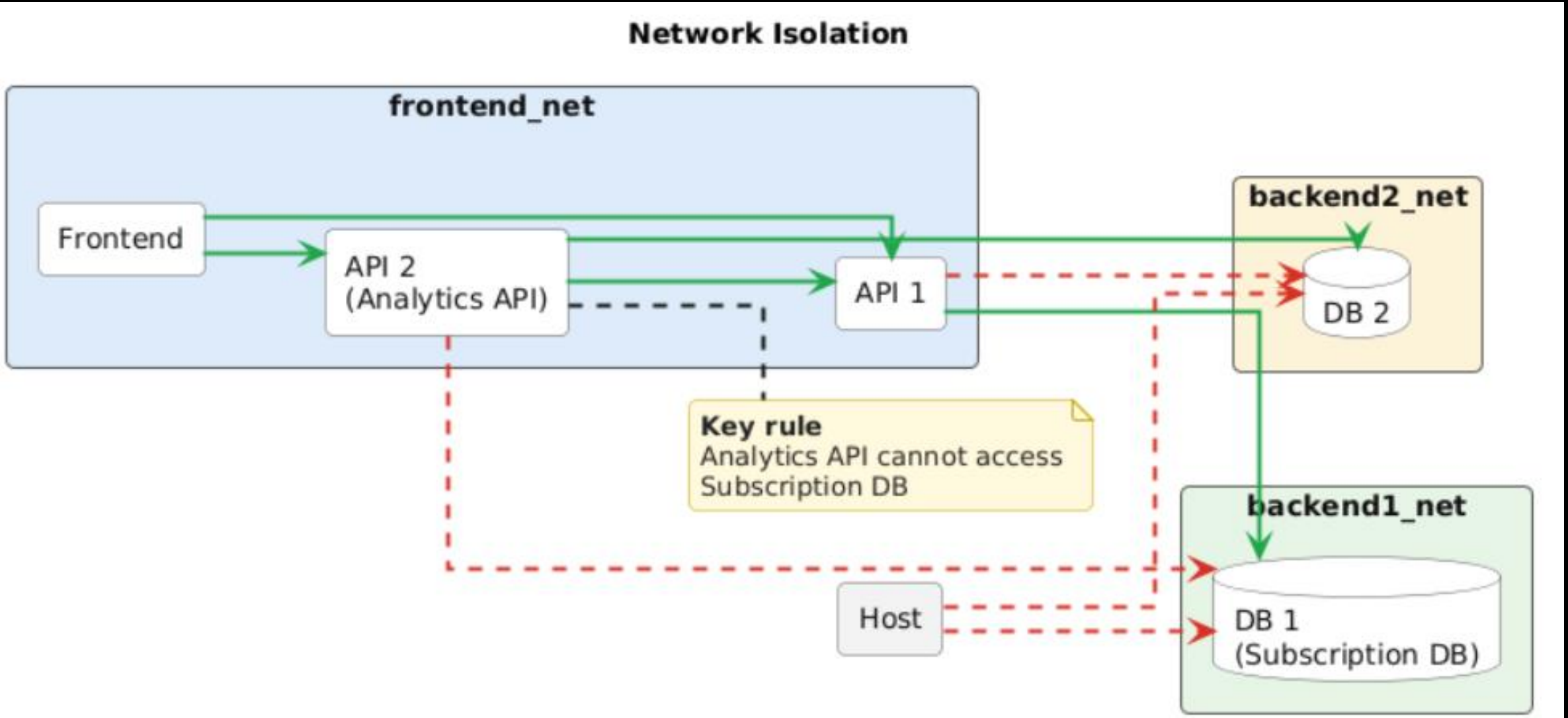
SERVICES BREAKDOWN



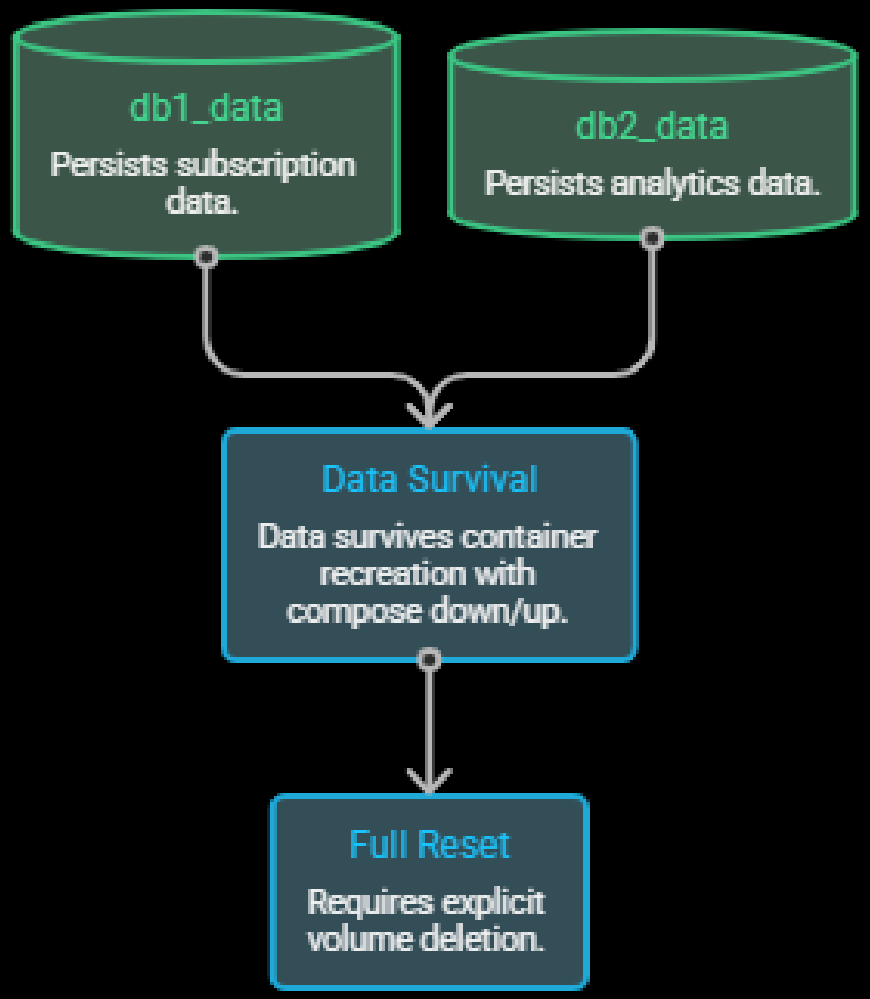
DATA FLOW


User Creates Subscription
User initiates subscription process in frontend
Frontend Sends Request
Frontend transmits subscription request to api1
Api1 Persists Data
Api1 stores subscription data in db1
Frontend Requests Analytics
Frontend requests analytics data from api2
Api2 Fetches Data
Api2 retrieves subscription data from api1
API
Api2 Computes Totals
Api2 calculates subscription totals
Api2 Stores Artifacts
Api2 saves analytics artifacts in db2
API
Api2 Returns Results
Api2 sends analytics results back to frontend

NETWORK ISOLATION



DATA PERSISTENCE



FRONTEND AND HOT RELOAD

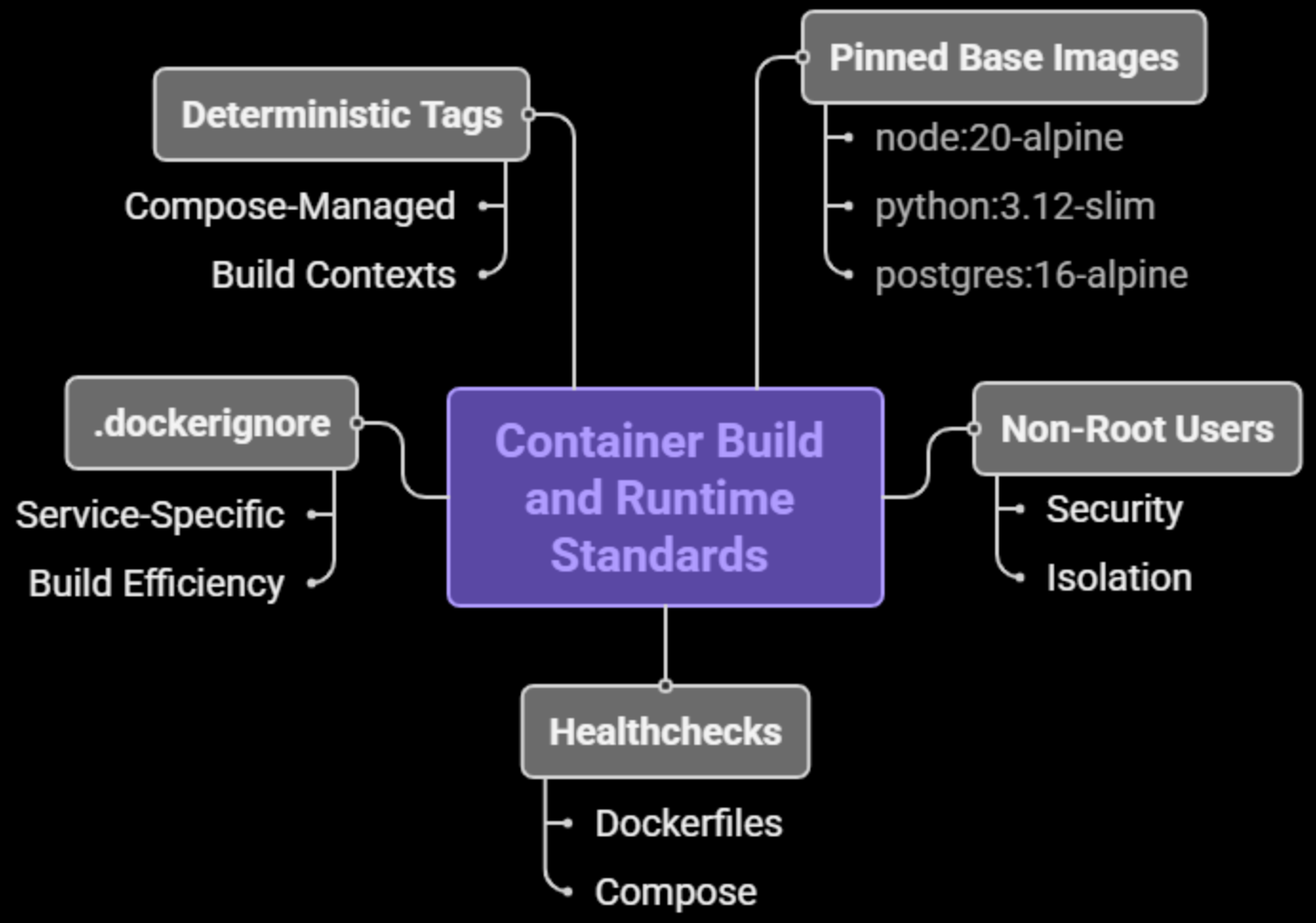
Frontend source directory is bind-mounted into container.

Vite dev server runs in container and serves on 5173.

File edits in source are reflected without rebuilding image.

frontend_node_modules volume avoids host/container dependency conflicts.

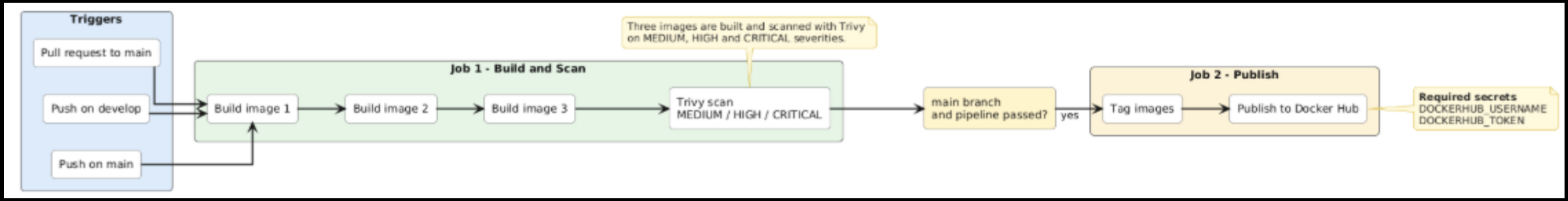
DOCKERIZATION STRATEGY



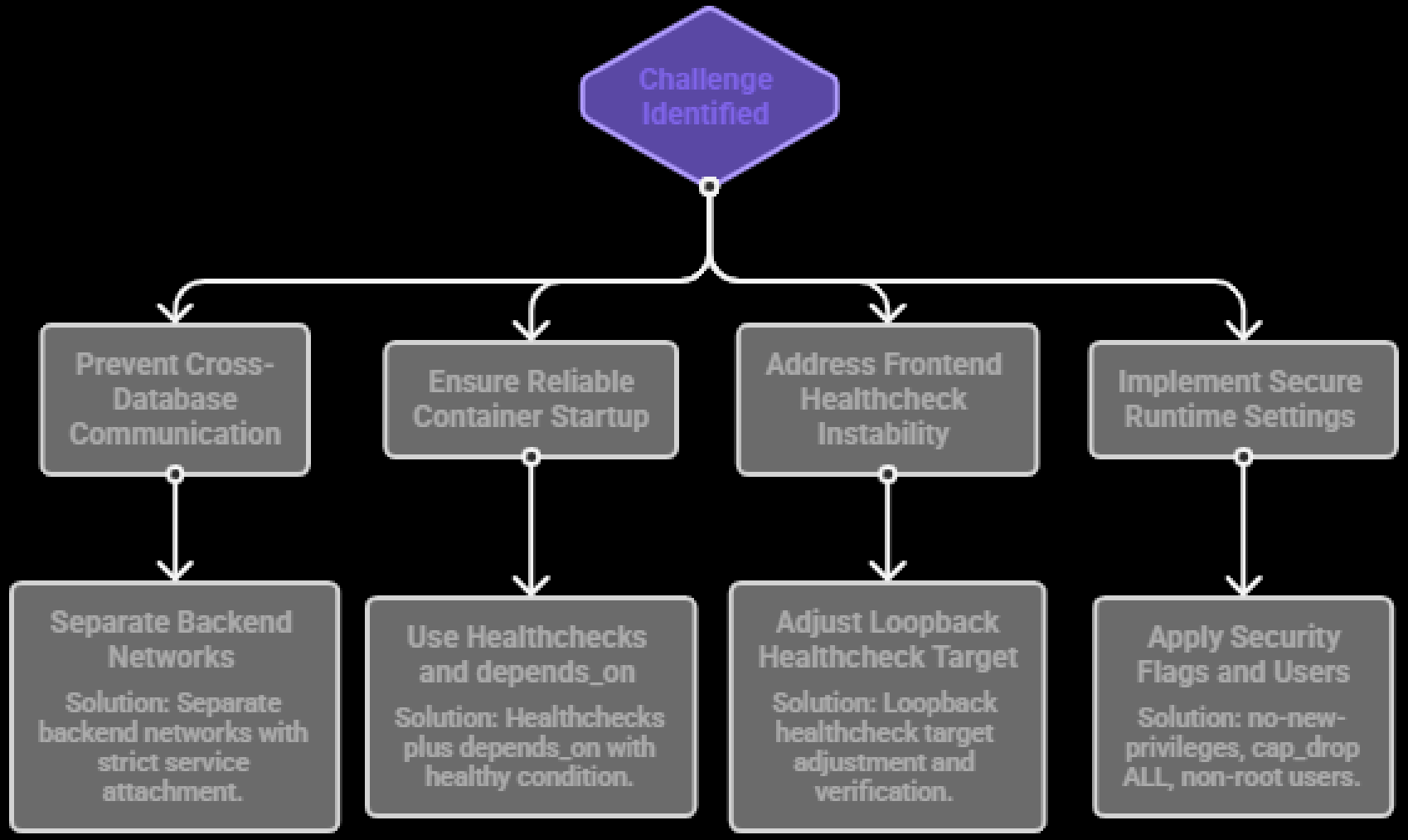
SECURITY AND BEST PRACTICES

- no-new-privileges enabled for app services.
- Linux capabilities dropped with cap_drop ALL.
- init enabled for signal handling and process reaping.
- pids_limit set for app containers.
- Databases not exposed on host ports.
- Trivy scanning in CI enforces vulnerability gates.

CI/CD PIPELINE



CHALLENGES AND SOLUTIONS



IMPROVEMENTS AND NEXT STEPS

API Authentication

Implement robust authentication and authorization for API access.

Database Migrations

Introduce database migrations for controlled and versioned schema evolution.

Reverse Proxy

Set up a reverse proxy with TLS termination for secure and efficient traffic management.

Observability Stack

Integrate an observability stack for metrics, logs, and traces to monitor application performance.

Test Automation

Implement test automation to ensure isolation and persistence assertions for reliable application behavior.

Image Optimization

Further optimize images using multi-stage runtime trimming for reduced size and faster deployment.

Made with Napkin

CONCLUSION

Demo-Readiness
The system is prepared for demonstration and maintenance
Clear Architecture
The system's structure is easy to understand and follow
System Quality
CI Pipeline
Automated processes ensure reliable delivery and vulnerability control
Isolation and Persistence
Data and processes are kept separate and data is stored reliably
Docker Security
Container security measures are in place

MERCI – THANKS YOU